



ScopeX Advanced Oscilloscope

Setup Manual

Driver Installation & Host Software Framework

ScopeX Integrated Laboratory Framework

- Dual-Channel Digital Storage Oscilloscope (DSO)
- Integrated DDS Analog Signal & Digital Frequency Generator
- Fixed +5VDC and High-Precision Bipolar $\pm 12V$ Dual Rail Supply
- Cross-Platform Native GUI Deployment (Windows, macOS, Linux)

Revision Tracking: v3.0

Supported Architecture Nodes: Windows 10/11 · macOS Sierra+ · Linux (Ubuntu 20.04+)

Confidential — For internal, industrial, and educational use

Contents		2.2.7 Launch GUI and Check	10
1 Introduction	2	2.3 macOS Setup Procedure	11
1.1 Cross-Platform Core Infrastructure	2	2.3.1 Install the Application Bundle	12
1.2 Kit Components Inventory	2	2.3.2 Bypass Gatekeeper Execution Restrictions	13
2 System Setup and Configuration	3	2.3.3 Connect Hardware and Verify Serial Node	14
2.1 Initial Physical Layout Logic	3	2.3.4 Launch GUI and Connect	15
2.2 Windows Setup Procedure	3	2.4 Linux Setup Procedure	16
2.2.1 Install Application and Drivers	4	2.4.1 Install FUSE (AppImage Runtime Dependency)	16
2.2.2 Open Executable	5	2.4.2 Assign Program Execution Flags	17
2.2.3 Connect Scopex and Find Device	6	2.4.3 Launch and Verify ACM Endpoint Connection .	17
2.2.4 Update Driver	7	3 Hardware Architecture	19
2.2.5 Install Driver	8	3.1 Main Board Overview	19
2.2.6 Verify COM Port Recognition	9	3.2 Board Features	20

1 Introduction

Overview: The Scope-X reference hardware system from Bumblebee Instruments relies on an onboard ATxmega processor executing a standard USB Communications Device Class (CDC-ACM) profile.
Note: This manual now contains setup instructions for Windows, macOS, and Linux host platforms.

This manual details the procedures required to flash, map, and bind host drivers to the Scope-X GUI interface application framework.

Depending on your structural target host platform (Windows, macOS, or Linux), device node acquisition requires specific local terminal access privileges. Users are advised to retrieve the structural workspace bundle containing the host executable binaries and the verbatim archive file named `Drivers.zip` before proceeding with initialization strings.

1.1 Cross-Platform Core Infrastructure

- **Windows 10/11 Deployment:** Utilizes inf-driven virtual structural mapping to allocate system terminal objects.
- **macOS Architectures:** Operates via the built-in communication class system layer, utilizing Gatekeeper pathway authorization.
- **Linux Environments:** Relies on kernel-level `cdc_acm` modules requiring structural group permission modifications.

1.2 Kit Components Inventory

Table 1: System Hardware Packaging Profile

Component Description			Interface Specification	Quantity
Scope-X Board	Main Processing		ATxmega Core / Analog Signal Mesh Layer	1
High-Speed	Comm-Link	Cable	Custom Shielded USB-B Link Cable	1

2 System Setup and Configuration

Important Deployment Rules: Observe clean Electrostatic Discharge (ESD) procedures when manipulating the processing architecture. Ensure the host processing terminal environment satisfies the cross-platform dependencies listed below.

2.1 Initial Physical Layout Logic

1. Unpack the structural hardware component stack and evaluate it against the packaging layout documentation.
2. Extract the software workspace folder and the drivers file onto a verified local storage matrix tracking route.
3. Provide the hardware block with proper power via the tracking cable. Verify active LED status loops.
4. Secure the USB-B link to target structural host ports to register internal endpoints.

Host Terminal System Requirements

- **Windows Platform:** Windows 10 / 11 (x64 Architecture)
- **Apple Platform:** Intel / Apple Silicon (M1/M2/M3) macOS 10.12+
- **Linux Open-Source:** Kernel 5.4+ (Ubuntu 20.04 LTS or equivalent)
- **Physical Port Allocation:** Dedicated USB 2.0/3.0 Type-A/Type-C Slot

2.2 Windows Setup Procedure

2.2.1 Install Application and Drivers

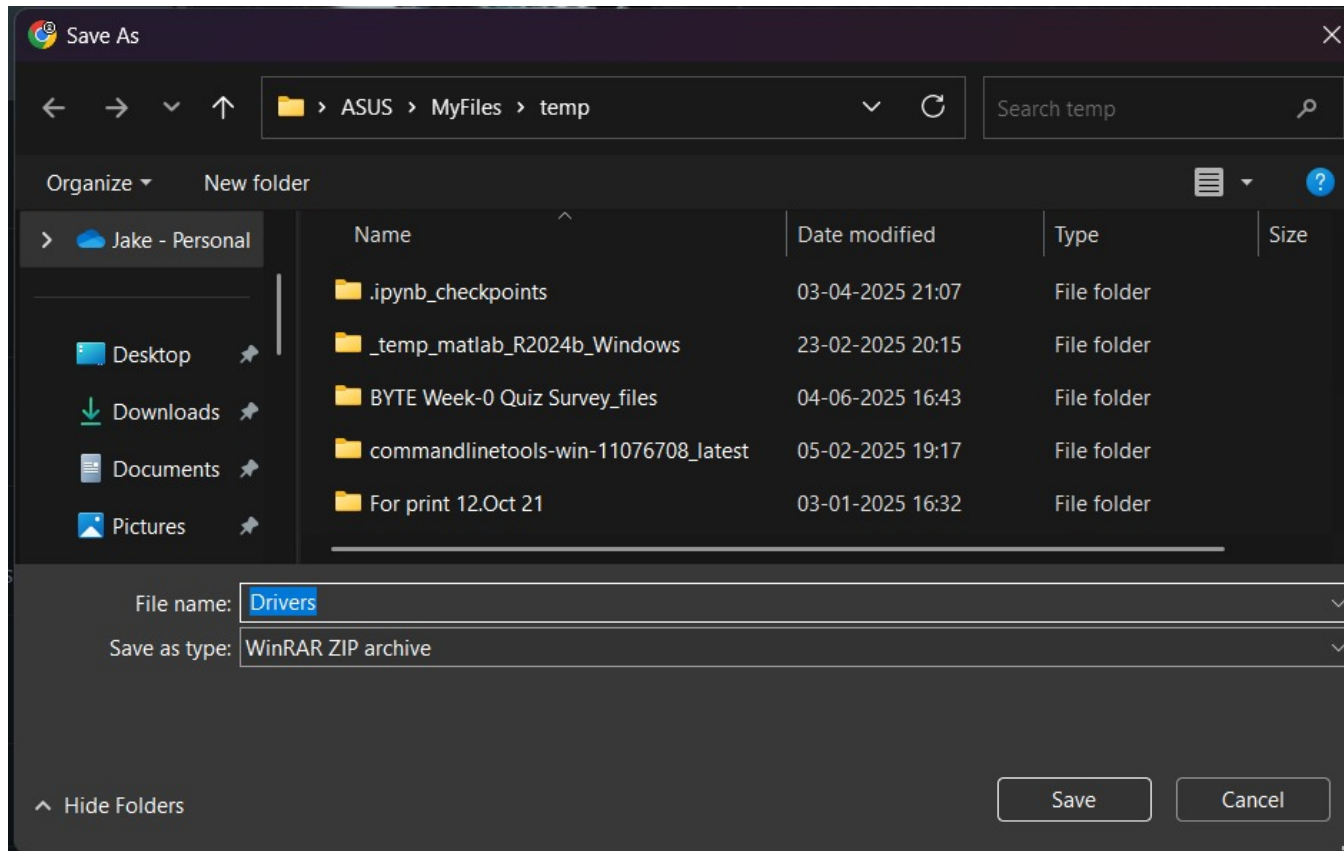


Figure 1: Locate your workspace bundle. Save the verbatim **Drivers.zip** archive and extract its contents to your preferred system directory alongside the main executable binary.

2.2.2 Open Executable



Figure 2: Open the host executable application. Select 'Run anyway' if prompted by the Windows SmartScreen security filter.

2.2.3 Connect Scopex and Find Device

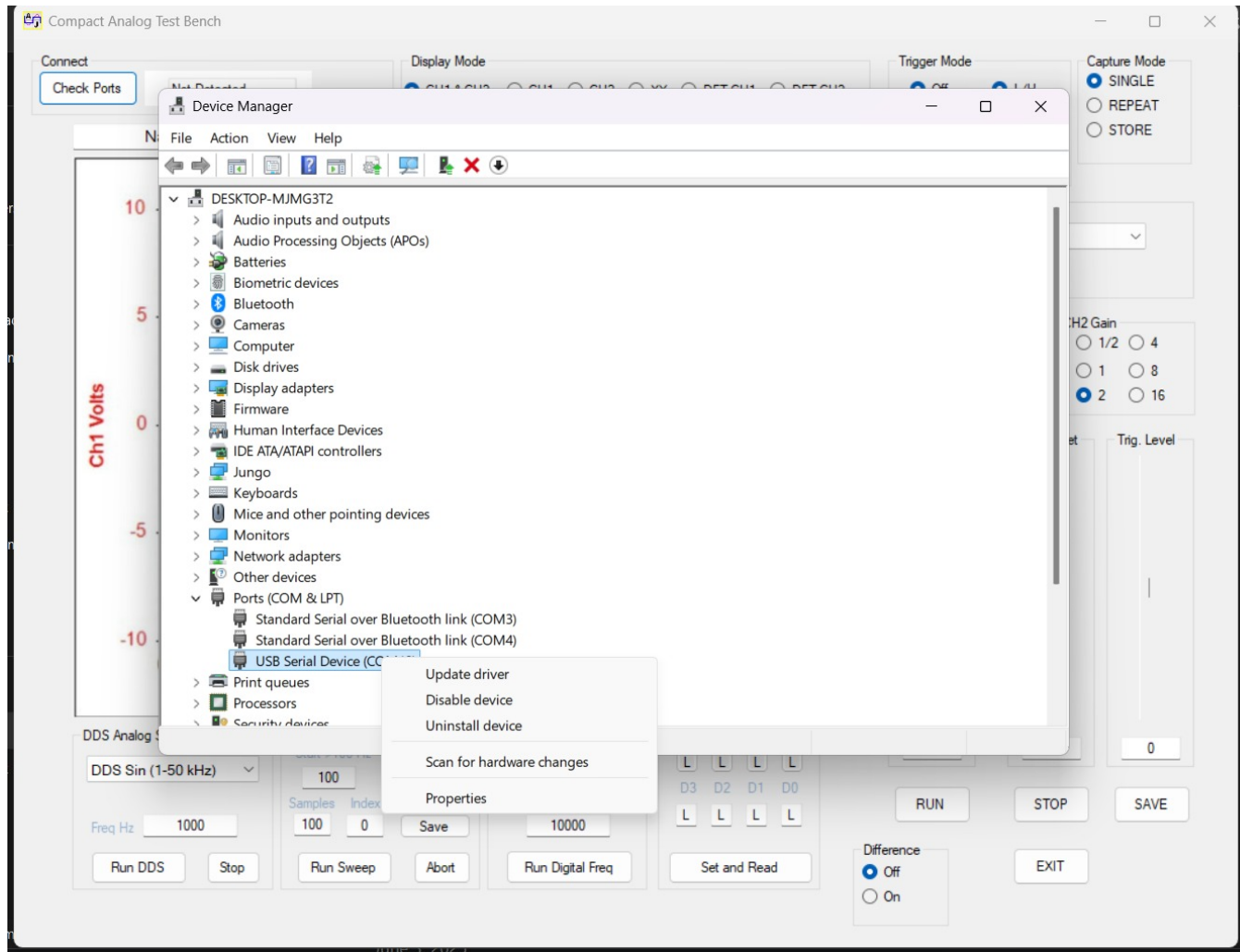


Figure 3: Interface the core board via USB. Open Device Manager and locate the unmapped 'USB Serial Device' entry inside the Ports (COM & LPT) categorization path.

2.2.4 Update Driver

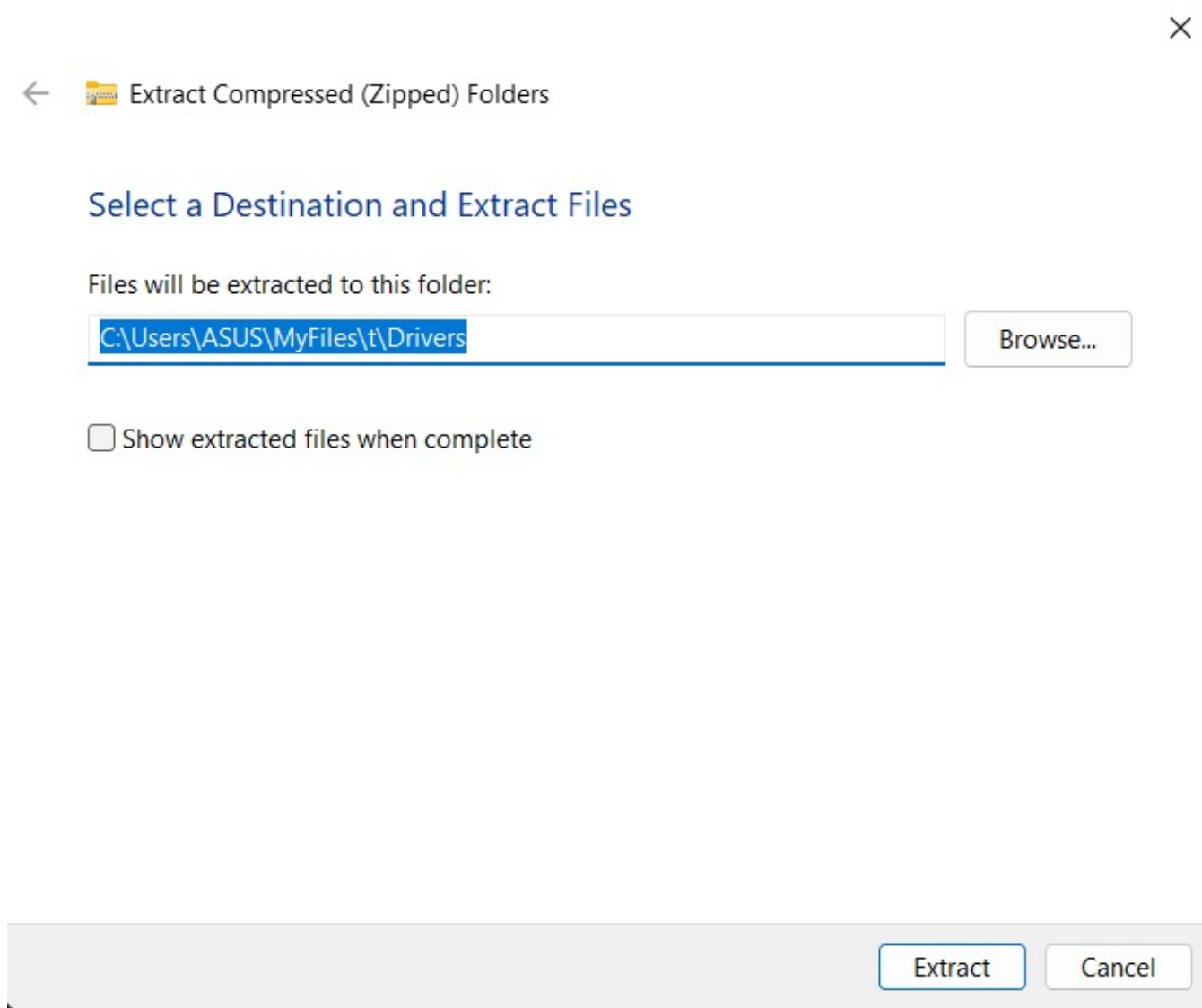


Figure 4: Right-click on the device entry, choose 'Update Driver', select the manual local lookup parameter, and navigate to the directory containing the uncompressed structure from `Drivers.zip`.

2.2.5 Install Driver

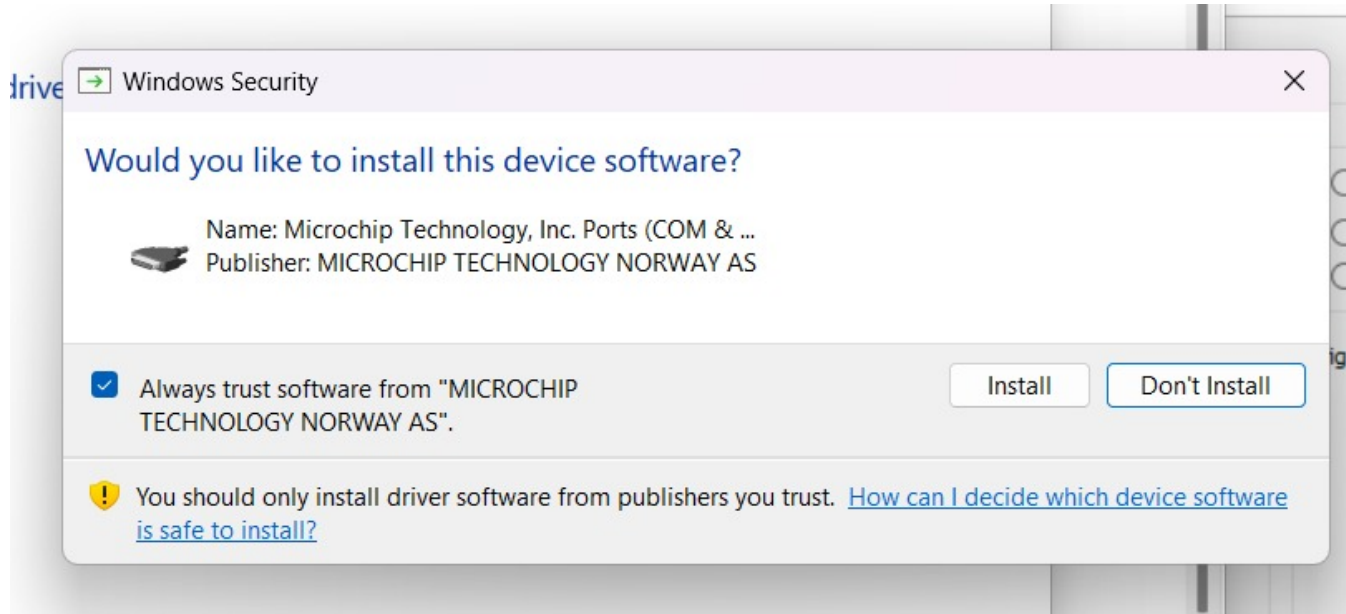


Figure 5: Accept any system prompts to authorize initialization. The tracking engine registers the inf parameters into host infrastructure indexes.

2.2.6 Verify COM Port Recognition

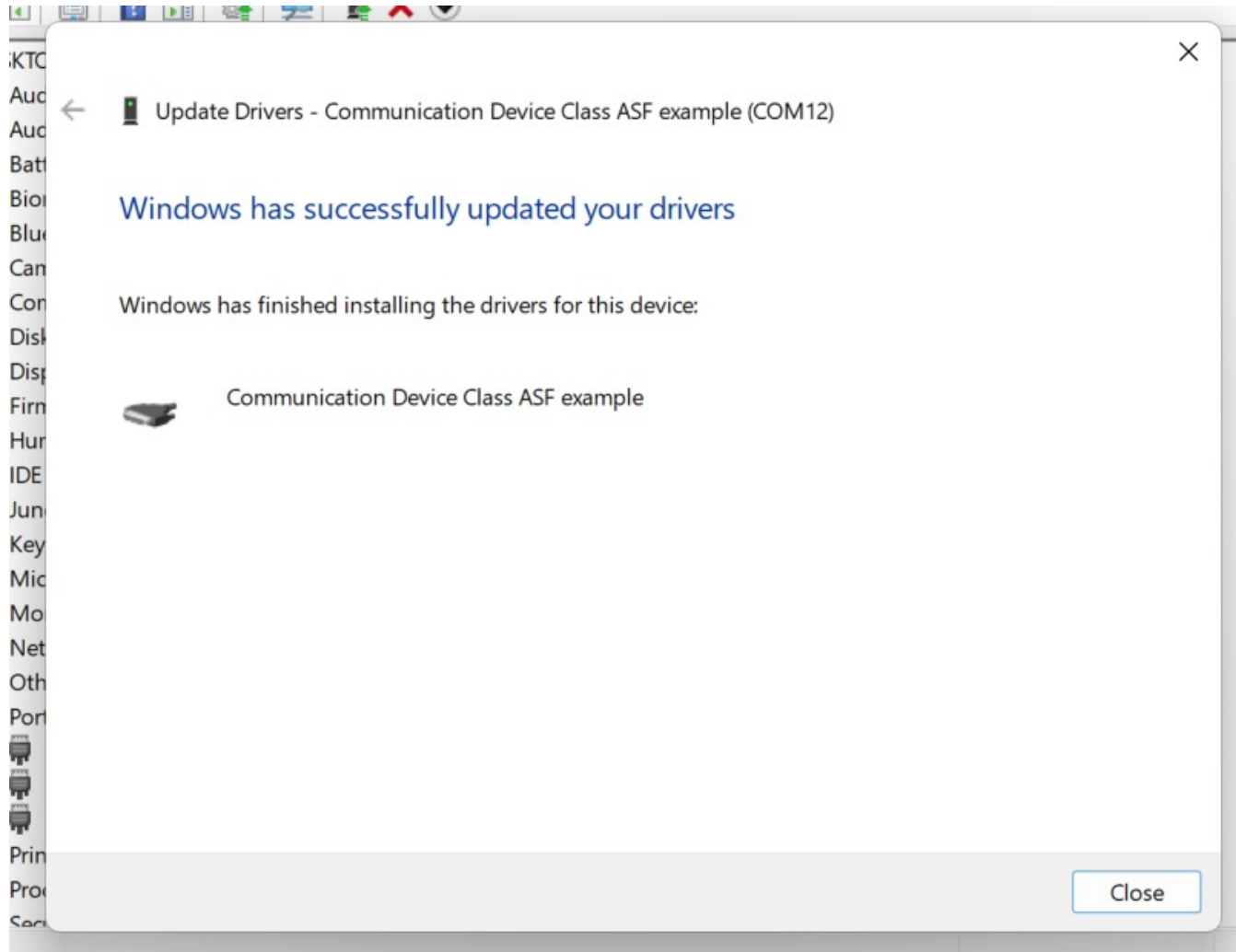


Figure 6: Verify that the system architecture successfully maps the target hardware, re-indexing it as a verified virtual communication endpoint node under an assigned COM identification block.

2.2.7 Launch GUI and Check

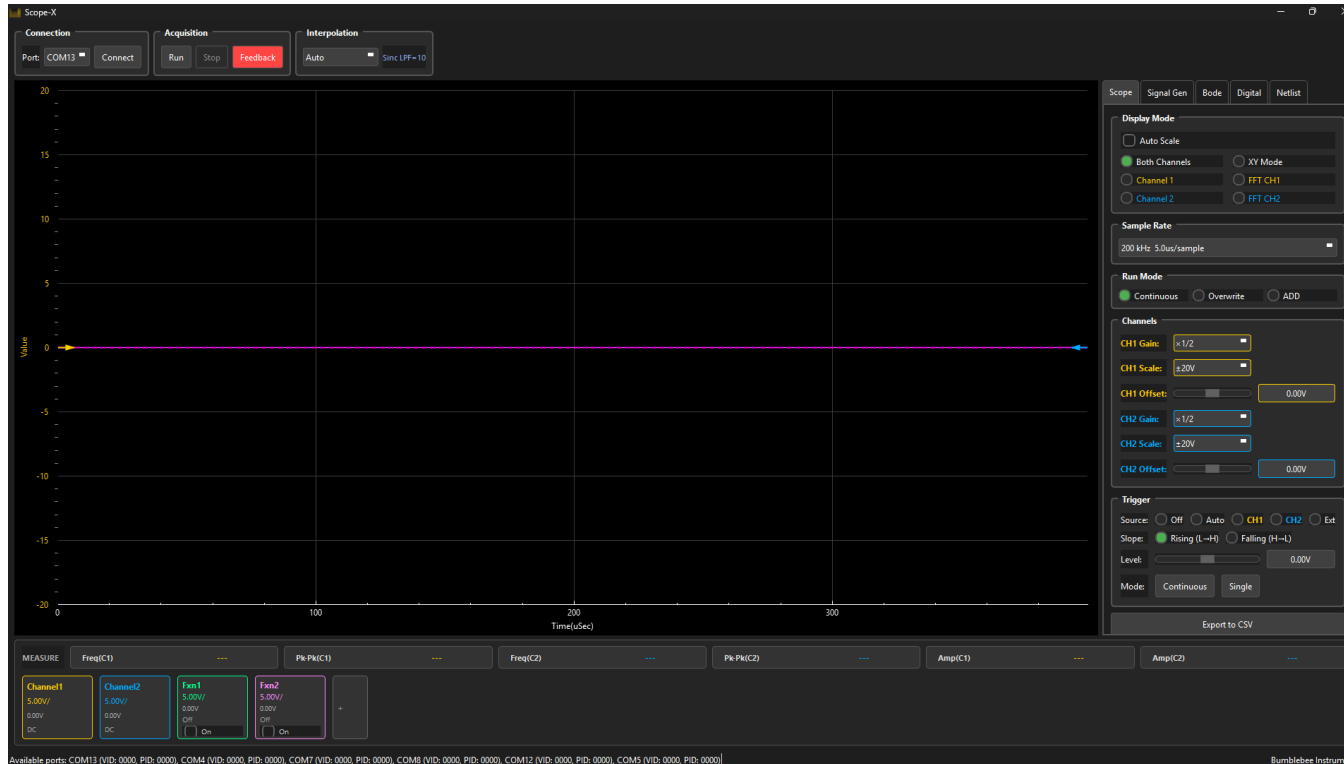


Figure 7: Once you launch the GUI, it tries to auto connect to the device, else you can try to Connect and now the device is connected to the GUI

2.3 macOS Setup Procedure

Modern macOS platforms automatically intercept ATxmega USB targets using the internal communications layer (the built-in CDC-ACM class driver). No separate driver package or `.kext` installation step is required on macOS – the Scope-X.app bundle ships with every Qt runtime framework (QtCore, QtGui, QtWidgets, QtSerialPort, QtSvg, QtPrintSupport, QtDBus) embedded directly inside `Contents/Frameworks`, so the application is fully self-contained.

Because the app is signed with an ad-hoc/developer certificate rather than notarized through the Mac App Store, macOS Gatekeeper will flag it as being from an “unidentified developer” on first launch. This is expected, and is resolved with a one-time manual override described

below.

1. Retrieve the **Scope-X.app** bundle (distributed as a `.dmg` disk image or a compressed `.zip` archive) and copy it into your local Applications directory.
2. Perform the one-time Gatekeeper authorization override.
3. Attach the Scope-X board to register the CDC-ACM serial descriptor.
4. Launch the app and bind to the assigned `/dev/cu.usbmodem*` node.

2.3.1 Install the Application Bundle



Figure 8: Mount the distributed **Scope-X.dmg** disk image (or unzip the **Scope-X.app** archive) and drag the **Scope-X.app** bundle into your **/Applications** folder. All required Qt frameworks are already embedded inside the bundle, so no separate runtime installation is needed.

2.3.2 Bypass Gatekeeper Execution Restrictions



Figure 9: On first launch macOS reports that **Scope-X** "cannot be opened because it is from an unidentified developer." Right-click (or Control-click) the app icon in Finder and choose **Open**, then confirm **Open** again in the dialog that follows. This whitelists the app permanently; subsequent launches proceed normally.

Alternative method: If the right-click override does not surface an Open option (common on macOS Ventura/Sonoma/Sequoia), go to **System Settings > Privacy & Security**, scroll to the Security section, and click **Open Anyway** next to the blocked Scope-X message, then authenticate with your password or Touch ID.

Terminal method (advanced): the quarantine flag can also be cleared directly from a terminal with:

```
1 xattr -cr /Applications/Scope-X.app
```

2.3.3 Connect Hardware and Verify Serial Node



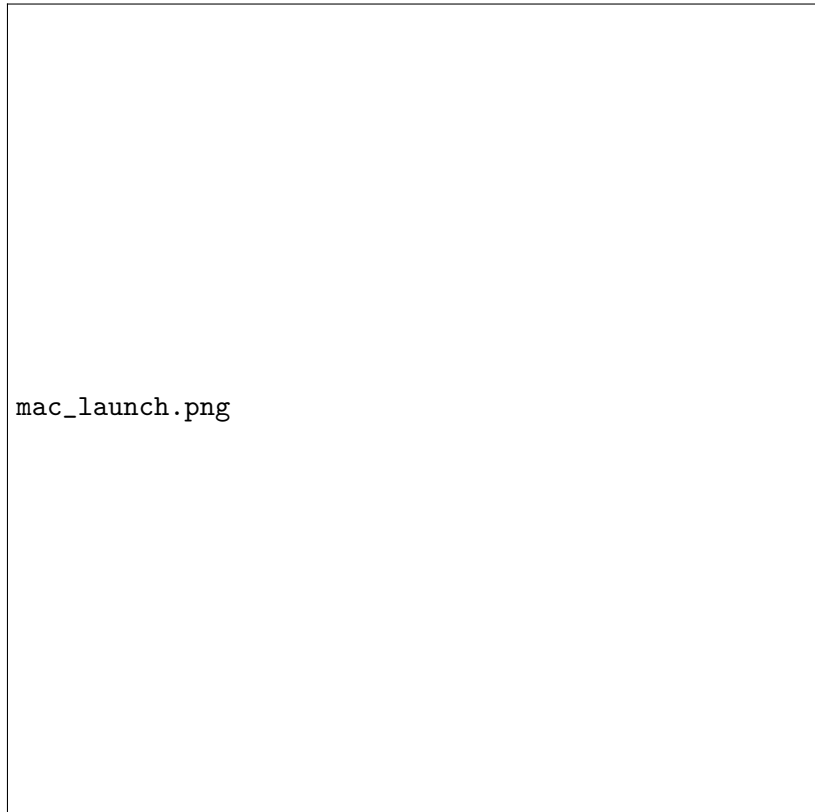
mac_verify.jpg

Figure 10: Connect the Scope-X board via the USB-B link cable. macOS auto-binds the device to its built-in CDC-ACM class driver with no user action required. Confirm the assigned device node by opening Terminal and running:

```
1 ls /dev/cu.usbmodem*
```

A path similar to `/dev/cu.usbmodem14201` confirms the board has enumerated correctly.

2.3.4 Launch GUI and Connect



mac_launch.png

Figure 11: Launch **Scope-X** from Launchpad or **/Applications**. The GUI attempts to auto-connect to the detected `cu.usbmodem*` node; if it does not, select the port from the connection dropdown and click **Connect**.

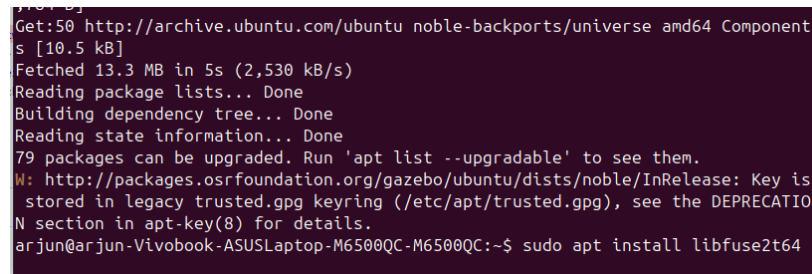
2.4 Linux Setup Procedure

Linux systems use the kernel-level `cdc_acm` module to capture ATxmega hardware links; this module is loaded automatically by the kernel and requires no manual installation. The Scope-X Linux build is distributed as a single portable AppImage binary with the Qt6 runtime (including `libQt6SerialPort.so.6`) bundled directly inside it, so no external Qt packages need to be installed.

While port mapping occurs immediately upon insertion, two host-side steps are still required: granting the AppImage execute permission, and adding your user account to the `dialout` group so the app can read/write the `/dev/ttyACM*` character device without root privileges.

- Install FUSE (required by most modern distributions to run AppImages).
- Mark the AppImage as executable.
- Add your user to the `dialout` group for serial port access.
- Launch the binary and verify the `/dev/ttyACM0` node.

2.4.1 Install FUSE (AppImage Runtime Dependency)



```
Get:50 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Component
s [10.5 kB]
Fetched 13.3 MB in 5s (2,530 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
79 packages can be upgraded. Run 'apt list --upgradable' to see them.
W: http://packages.osrfoundation.org/gazebo/ubuntu/dists/noble/InRelease: Key is
stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATIO
N section in apt-key(8) for details.
arjun@arjun-Vivobook-ASUSLaptop-M6500QC-M6500QC:~$ sudo apt install libfuse2t64
```

Figure 12: Ubuntu 22.04+/24.04 and several other modern distributions no longer ship `libfuse2` by default, which AppImages require to mount and run themselves. Open a terminal and install it:

```
1 sudo apt update
2 sudo apt install libfuse2t64 # or libfuse2 on older Ubuntu releases
```

No-FUSE fallback: if FUSE cannot be installed on your system, the AppImage can still be run directly by extracting it on demand:

```
1 ./Scope-X-x86_64.AppImage --appimage-extract-and-run
```

2.4.2 Assign Program Execution Flags

```
1 chmod +x Scope-X-x86_64.AppImage
```

```
1 sudo usermod -a -G dialout $USER
```

Log out and back in (or reboot) to reload your group membership – this step will not take effect in the current session.

2.4.3 Launch and Verify ACM Endpoint Connection

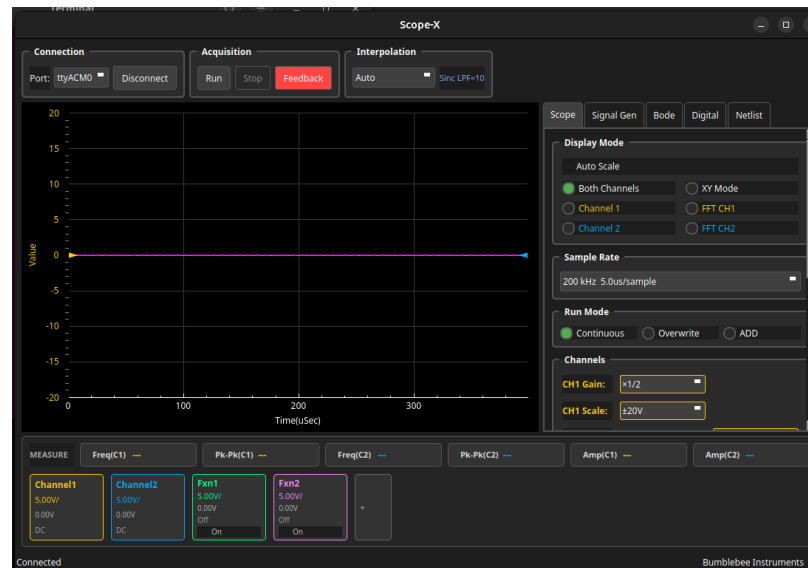


Figure 13: Connect the Scope-X board via USB, then launch the tool:

```
1 ./Scope-X-x86_64.AppImage
```

Confirm the kernel has enumerated the device by checking for the character node:

```
1 ls /dev/ttyACM*
```

A result such as `/dev/ttyACM0` confirms success. Inside the GUI, click **Check Ports** / **Connect** to establish the link.

Optional: Add a Desktop Launcher Entry

The AppImage embeds its own icon and `.desktop` descriptor. For a permanent launcher icon (menu/dock entry) without a full package manager integration tool, extract and install them manually:

```
1 ./Scope-X-x86_64.AppImage --appimage-extract
2 mkdir -p ~/.local/share/applications ~/Applications
3 cp squashfs-root/Scope-X.desktop ~/.local/share/applications/
4 cp squashfs-root/Scope-X.png ~/.local/share/icons/
5 mv Scope-X-x86_64.AppImage ~/Applications/
```

Then edit the `Exec=` and `Icon=` lines of the copied `.desktop` file to point at the final AppImage and icon paths. Alternatively, install a helper such as `AppImageLauncher` to automate this integration.

3 Hardware Architecture

3.1 Main Board Overview

The Scope-X main board features a modular architecture optimized for educational use and rapid prototyping.



Figure 14: Scope-X Main Board

3.2 Board Features

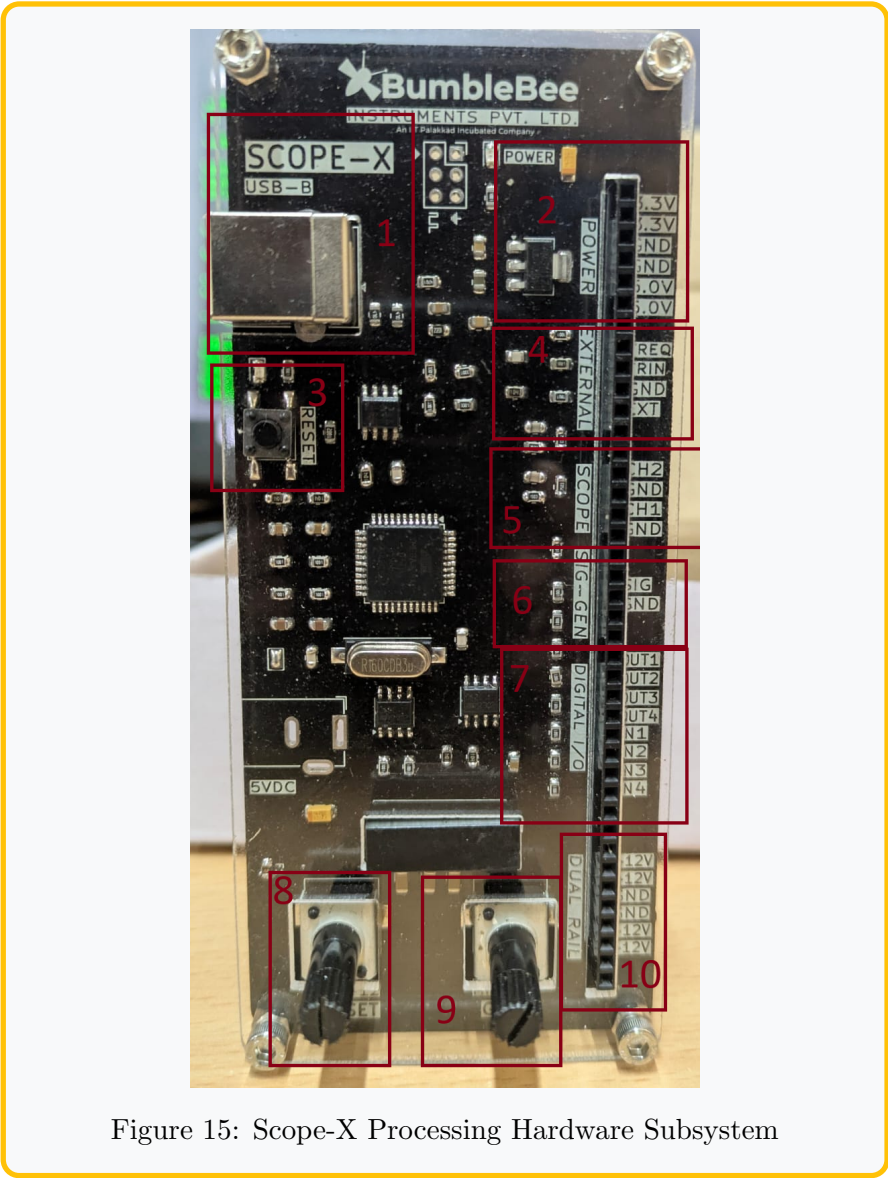


Figure 15: Scope-X Processing Hardware Subsystem

Table 2: System Mapping Allocation Table

Feature Assignment	Operational Use Case Parameters
1 USB-B Port	High-Speed Virtual COM Pipeline
2 Power Switch	Main Rail Isolation Management
3 Reset Link	Core Microcontroller Restart Matrix
4 External Bus	Accessory Processing Routing Matrix
5 Scope Inputs	Dual-Channel System Diagnostics Block
6 DDS Generator	Programmable Continuous Waveform Matrix
7 Digital I/O Block	4-Bit Capture and Stimulation Channels
8 Offset Adjust	Operational Tuning Block (-12V to +12V)
9 Gain Matrix	Precision Hardware Scaler Interface
10 Dual Rail Block	Bipolar Lab Reference Subsystem